IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Una red neuronal con solo NumPy

Teoría, práctica y, al final, material de consulta

Sprint 10 · S10_01 · teoría breve (20 min el 17/11), 40 min de práctica en clase y material de consulta

LA IDEA · LA NEURONA

Una neurona: sumar con pesos y comprimir

Tres pasos: multiplica cada entrada por un **peso**, suma todo y añade un **sesgo**, y pasa el resultado por una **función de activación**, un «interruptor suave».

Entrada	valor	peso	valor × peso
Antelación (cientos de días)	2,0	0,5	1,0
Desayuno incluido (1 = sí)	1,0	−0,6	−0,6
Suma + sesgo (0,2)			z = 0,6

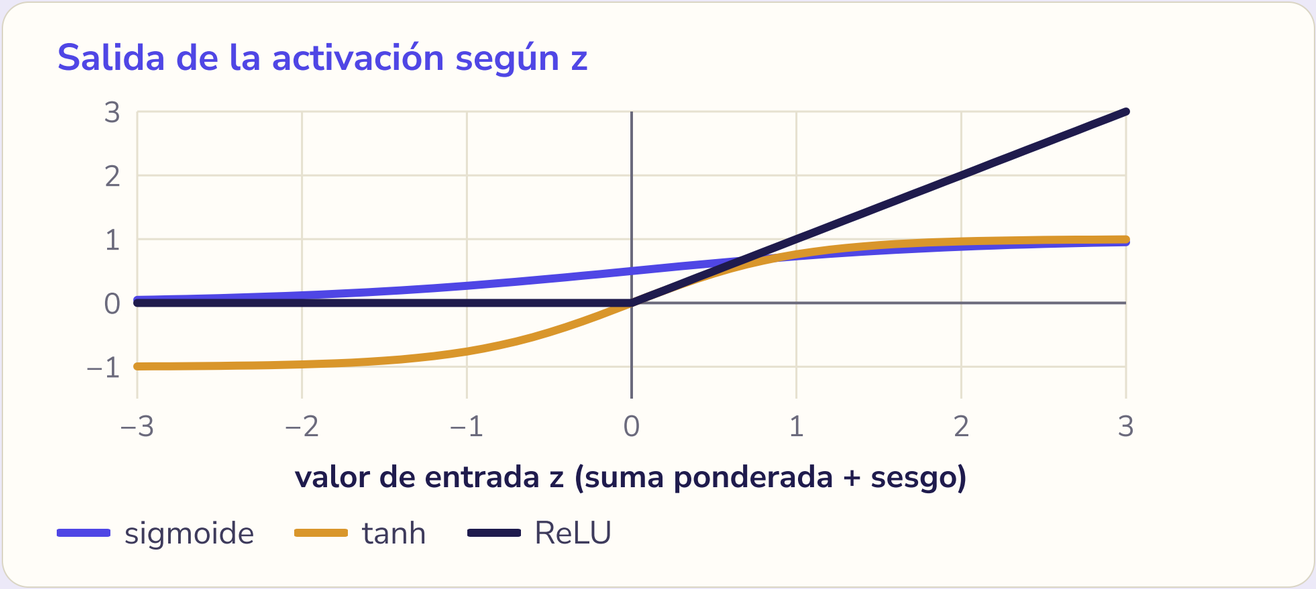
El sesgo

Es un **empujón inicial**: se suma aunque todas las entradas valgan 0.

0,646

sigmoide(0,6): la neurona da un **65 %** de probabilidad de cancelar.

Sin curvas, apilar capas no sirve de nada



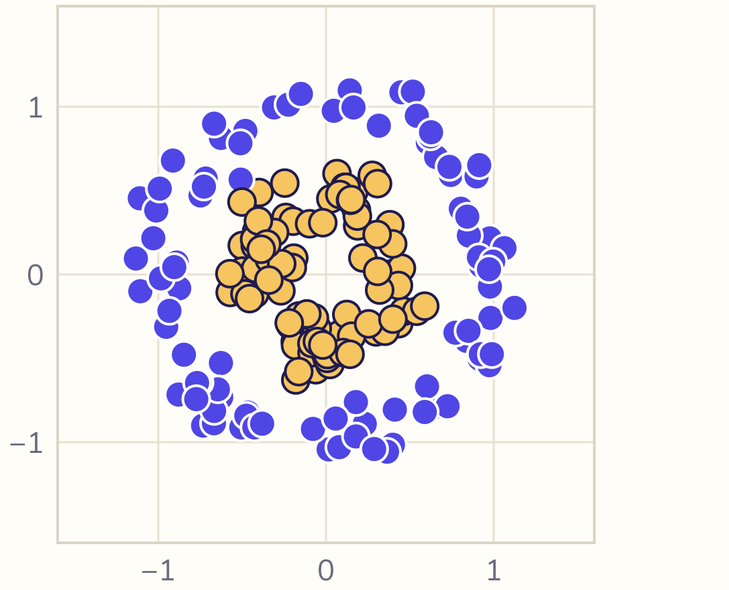
z	sigmoide	tanh	ReLU
−1,5	0,18	−0,91	0
0	0,50	0,00	0
2,5	0,92	0,99	2,5

Una suma de sumas ponderadas sigue siendo una suma ponderada: sin activación, la red es un modelo **lineal.**

EL PROBLEMA · DOS ANILLOS

Dos anillos que ninguna recta separa

Puntos de dos grupos, uno dentro del otro



Una recta no basta

Cualquier recta deja parte de cada anillo a cada lado: un modelo lineal acierta **como el azar**.

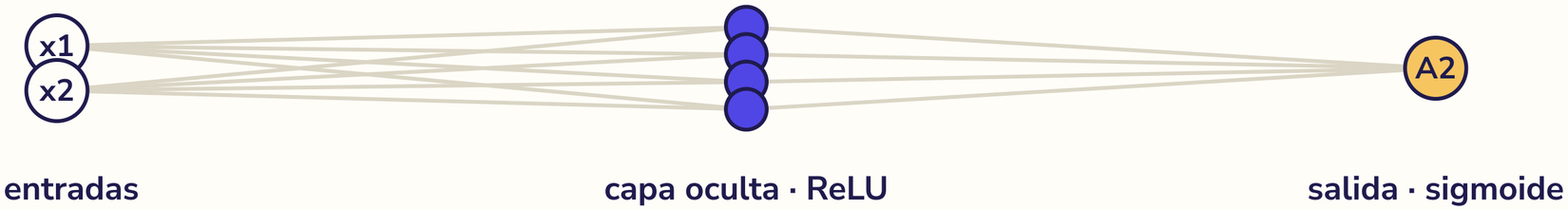
La red que vamos a construir

2 entradas → capa oculta (ReLU) → 1 salida (sigmoide).

APRENDER · HACIA DELANTE

De las entradas a la predicción

Una capa son muchas neuronas a la vez: se calcula con **una multiplicación de matrices**.

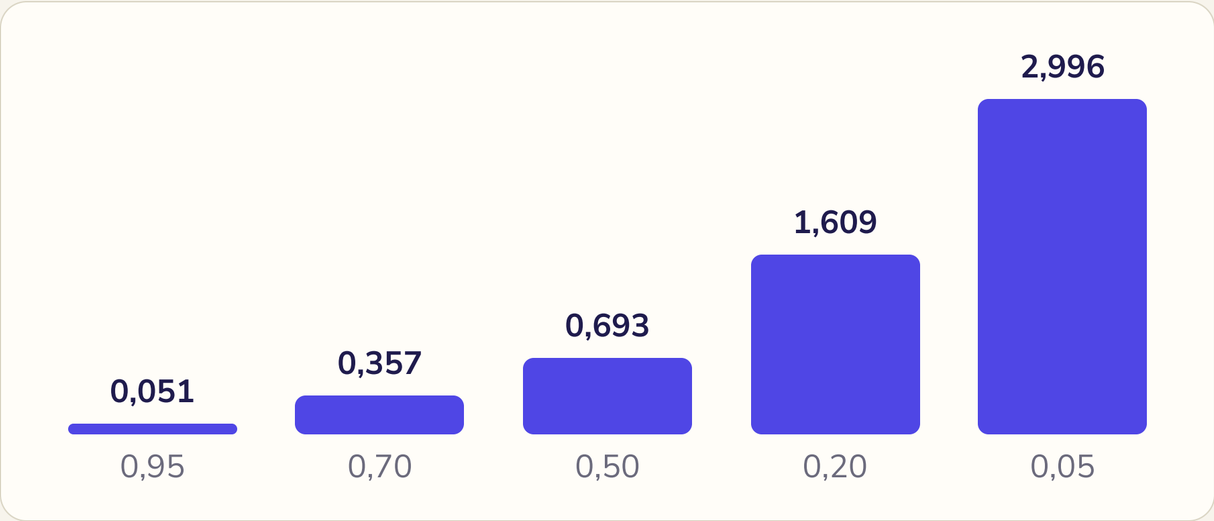


Paso	Qué calcula	Fórmula
1	Suma ponderada de la capa oculta	Z1 = X · W1 + b1
2	Activación	A1 = ReLU(Z1)
3	Suma ponderada de la salida	Z2 = A1 · W2 + b2
4	Probabilidad de ser del grupo 1	A2 = sigmoide(Z2)

APRENDER · MEDIR EL ERROR

La pérdida castiga fallar con seguridad

Entropía cruzada: mira la probabilidad que la red dio a la **respuesta correcta**. Menor es mejor.



Probabilidad a lo correcto → pérdida

0,5 (no saber nada) es ln 2 = 0,693.

Si acierta con bastante seguridad (0,95), la pérdida es **0,051**; si falla con seguridad (0,05), **2,996**.

Fallar con seguridad se castiga mucho más que acertar con seguridad se premia.

APRENDER · HACIA ATRÁS

El gradiente: ¿sube o baja el error?

Mini-red de juguete: 1 entrada, 1 neurona oculta (ReLU) y 1 salida (sigmoide). Una reserva con **x** = 2 que **sí cancela** (**y** = 1) y pesos w1 = 0,5, b1 = 0,0, w2 = 1,0, b2 = −0,5.

Hacia delante →	cálculo	valor
z1	2 · 0,5 + 0	1,00
a1 = ReLU		1,00
z2	1,00 · 1,0 − 0,5	0,50
a2 = sigmoide		0,622
error	−ln(0,622)	0,474

← Hacia atrás	cálculo	valor
dz2	a2 − y	−0,378
dw2 y **db2**	a1 · dz2 ; dz2	−0,378
dz1	dz2 · w2 · 1 (z1 > 0)	−0,378
dw1	x · dz1	−0,755
db1	dz1	−0,378

El error de la salida (a2 − y) viaja hacia atrás: todos los gradientes son negativos, así que subir los pesos baja el error.

APRENDER · UN PASITO

Un pasito en contra del gradiente

Parámetro	valor	gradiente	nuevo = valor − 0,1 · gradiente
w1	0,500	−0,755	0,576
b1	0,000	−0,378	0,038
w2	1,000	−0,378	1,038
b2	−0,500	−0,378	−0,462

La tasa de aprendizaje

Es el **tamaño del pasito**: aquí vale 0,1. Se **resta** porque el gradiente señala hacia dónde sube el error.

0,474 → 0,380

El error de esta reserva baja tras **un solo** paso.

adelante → error → atrás → actualizar. Repetirlo con todos los datos es una época.

PRÁCTICA · CUADERNO S10_01

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Cargar las herramientas	Clase
Ejercicio 1	La función **neurona**	Clase
Ejercicio 2	ReLU y su derivada	Clase
Ejercicio 3	Un paso de descenso del gradiente	Clase
Para leer	La clase **RedNeuronal** y su prueba de gradientes	Casa
Ejercicio 4	Exactitud de la red y mejora sobre el modelo lineal	Casa
Mini-reto	Una red con tasa de aprendizaje 0,01	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**: ejercicios 1 a 4 y mini-reto, con su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	La función neurona	11 y 12
2	ReLU y su derivada	13 y 14
3	Un paso de gradiente	15 y 16
Ejemplos	La red completa y entrenar con 1, 2 y 8 neuronas	17 y 18
4	Exactitud y mejora sobre el lineal	19 y 20
Mini-reto	La tasa de aprendizaje	21 y 22

CONSULTA · EJERCICIO 1

La función neurona (ejercicio 1)

```
def neurona(x, pesos, sesgo):
    return sigmoide(np.dot(x, pesos) + sesgo)
neurona(np.array([1.5, 1.0]), np.array([0.8, 1.2]), -1.0)    # 0.802
neurona(np.array([0.0, 0.0]), np.array([5.0, 5.0]), 0.0)     # 0.5
```

1,4

z = suma ponderada + sesgo

0,802

salida de la neurona

La neurona «cree» que esa reserva tiene un 80 % de probabilidad de cancelar.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer la neurona (ejercicio 1)

np.dot(x, pesos)

Multiplica cada entrada por su peso y **suma**: 1,5 · 0,8 + 1,0 · 1,2 = 2,4. Con * saldría un vector, no un número.

El sesgo

Se suma **después**: 2,4 − 1,0 = 1,4. Es el empujón inicial de la neurona, aunque todas las entradas valgan 0.

0,802 ≈ 80 %

sigmoide(1,4): la neurona estima un **80 %** de probabilidad de cancelar esa reserva.

Con entradas 0 y sesgo 0

Sale exactamente **0,5** (**sigmoide(0)**): sin información, la neurona está indecisa. Es el segundo **assert**.

CONSULTA · EJERCICIO 2

ReLU y su derivada (ejercicio 2)

```
def relu(z):
    return np.maximum(0, z)
def relu_derivada(z):
    return (z > 0).astype(float)
relu(np.array([-2.0, 0.0, 3.0]))              # [0. 0. 3.]
relu_derivada(np.array([-2.0, 0.0, 3.0]))     # [0. 0. 1.]
```

0 · 0 · 3

relu de −2, 0 y 3

0 · 0 · 1

relu_derivada

Una neurona ReLU con z ≤ 0 está apagada: no aprende en esa muestra.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer ReLU y su derivada (ejercicio 2)

np.maximum(0, z)

Funciona con **arrays**; el **max** de Python no. Anula los negativos y deja pasar los positivos.

(z > 0).astype(float)

La derivada devuelve **números** (0. y 1.), no booleanos. Vale 1 solo si **z > 0**, estrictamente.

z = −2, 0 y 3

relu: **0, 0 y 3**. **relu_derivada**: **0, 0 y 1**. Por convención, en **z = 0** la derivada es 0.

Neurona ReLU apagada

Con **z ≤ 0** su derivada es 0: **no aprende en esa muestra**, porque el error no puede viajar hacia atrás por ella.

CONSULTA · EJERCICIO 3

Un paso de descenso del gradiente (ejercicio 3)

```
def paso_gradiente(parametros, gradientes, tasa):
    return {k: parametros[k] - tasa * gradientes[k] for k in parametros}
_p = {"W1": np.array([1.0, 2.0]), "b1": np.array([0.5])}
_g = {"W1": np.array([0.5, -1.0]), "b1": np.array([1.0])}
paso_gradiente(_p, _g, 0.1)    # W1 → [0.95 2.1] · b1 → [0.4]
```

[0,95 · 2,1]

W1 nuevo

[0,4]

b1 nuevo

Nuevo = valor – tasa · gradiente, y sin tocar el diccionario original.

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer el paso de gradiente (ejercicio 3)

Se resta, no se suma

El gradiente apunta hacia donde el error **sube**: para bajarlo se va en sentido contrario.

Un diccionario nuevo

La comprensión **{k: … for k in parametros}** **construye otro**: el original no cambia (**W1[0]** sigue valiendo 1,0). Es el segundo **assert**.

Resultado de la prueba

W1 = [0,95 · 2,1]: 1,0 − 0,1 · 0,5 = 0,95 y 2,0 − 0,1 · (−1,0) = 2,1. Y **b1 = [0,4]**.

Una tasa enorme

Los pasos serían tan grandes que **se saltaría el mínimo** y el error subiría en vez de bajar. Se ve en el mini-reto.

La red completa y su prueba de gradientes

```
red_test = RedNeuronal(2, 4)
red_test.adelante(X_ent); g = red_test.atras(X_ent, y_ent)
round(float(g["W1"][0, 1]), 6)    # 0.002669
round(float(numerico), 6)         # 0.002669
```

0,002669

gradiente de **W1[0, 1]** por **retropropagación**

0,002669

gradiente **numérico** (mover el peso ±0,00001)

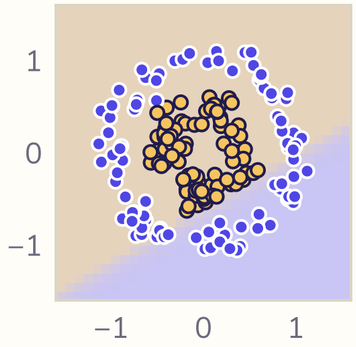
adelante predice; **atras** calcula cuánto corregir cada peso; **entrenar** repite adelante → atrás → actualizar.

Coinciden: la propagación hacia atrás está bien programada.

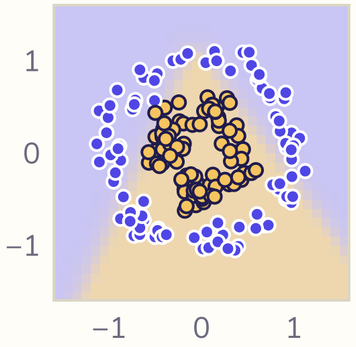
CONSULTA · SECCIÓN 6 · ENTRENAR CON 1, 2 Y 8 NEURONAS

Más neuronas, una frontera más fina

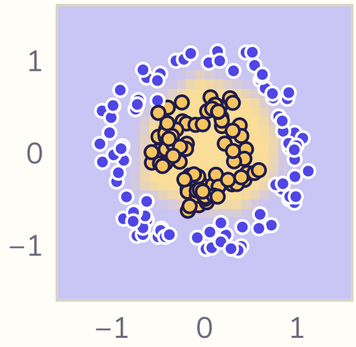
1 neurona oculta · 65,6 %



2 neuronas ocultas · 77,2 %



8 neuronas ocultas · 100 %



Fondo dorado: la red predice «grupo 1»; fondo lavanda: «grupo 0». Los puntos son los de validación.

```
for n_ocultas in (1, 2, 8):
    red = RedNeuronal(2, n_ocultas).entrenar(X_ent, y_ent, epocas=1500,
                                             tasa=0.3, X_val=X_val, y_val=y_val)   # validación: 0.656 · 0.772 · 1.000
```

CONSULTA · EJERCICIO 4

Exactitud de la red y mejora sobre el lineal

```
exactitud_val = (mejor.predecir(X_val) == y_val).mean()              # 1.0
mejora_sobre_lineal = exactitud_val - lineal.score(X_val, y_val)     # 0.506
```

1,0

exactitud_val (red de 8 neuronas)

0,506

mejora_sobre_lineal

De acertar el 49 % con una recta a acertar el 100 %: 51 puntos porcentuales más.

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer la mejora sobre el lineal (ejercicio 4)

mejor.predecir(X_val) == y_val

Compara con las etiquetas de **validación** (180 puntos que no usó para entrenar); **.mean()** da la fracción de aciertos.

lineal.score(X_val, y_val)

La exactitud del modelo lineal de antes (**0,494**). Si se midiera con otros datos o con **lineal** en vez de **mejor**, saldría otro valor.

1,0 y 0,506

exactitud_val = 1,0; **mejora_sobre_lineal** = 1,0 − 0,494 = **0,506**: 51 puntos porcentuales.

¿Tan fácil en la vida real?

No: los anillos son un problema de **juguete**, con 2 variables y poco ruido. Con datos reales no se llega al 100 %.

CONSULTA · MINI-RETO

La tasa de aprendizaje manda (mini-reto)

Tasa (4 neuronas, 1 500 épocas)	Pérdida de validación: época 0 → 1 499	Exactitud de validación
0,3	0,716 → 0,025	100 %
0,01	0,721 → 0,621	60,6 %
50	1,122 → 5,944 (oscila)	50,0 %

Muy pequeña: lenta

Con 0,01 la red apenas ha empezado a bajar la pérdida tras 1 500 épocas.

Enorme: inestable

Con 50 los pesos saltan de un lado a otro: la pérdida **se dispara** y no aprende.

```
red_lenta = RedNeuronal(2, 4).entrenar(X_ent, y_ent, epocas=1500,
                                       tasa=0.01, X_val=X_val, y_val=y_val)
exactitud_lenta = (red_lenta.predecir(X_val) == y_val).mean()   # 0.606
```

CONSULTA · ACLARACIONES · MINI-RETO

Leer la tasa de aprendizaje (mini-reto, casa)

tasa=0.01

El parámetro que hay que cambiar (en las otras redes era 0,3). Con otra tasa, el segundo **assert** se queja.

epocas=1500

El primer **assert** exige que la última época registrada sea la **1499**.

0,606

Pasos tan pequeños que en 1 500 épocas la red apenas avanza: pérdida final ≈ 0,62 (la red buena, 0,04).

Tasa 50

Probado: **no aprende** (exactitud ≈ 0,50) y la pérdida **sube** a ≈ 6. La tasa se elige probando valores en escala logarítmica y mirando la curva.